



# Programming languages Java

## Data representation

Kozsik Tamás



# Object-oriented programming

## Object-oriented programming (OOP)

- Object
- Class
- Abstraction
  - ◇ Encapsulation
  - ◇ Information hiding
- Inheritance
- Subtyping, subtype polymorphism
- Overriding, dynamic binding



# Encapsulation: object

Data and operations that can be performed on it (similar to structs in other languages)

- “Point” object
- “Rational number” object
- “List” object
- “Bank customer” object

```
p.x = 0;  
p.y = 0;  
p.move(3,5);  
System.out.println(p.x);
```



# Method

```
// in Java
```

```
p.x = 0;
```

```
p.y = 0;
```

```
p.move(3, 5);
```

```
// in languages that do not have objects (like C)
```

```
p.x = 0;
```

```
p.y = 0;
```

```
move(p, 3, 5);
```



# Class

A type in which similar objects belong

- “Point” class
- “Rational number” class
- “List” class
- “Bank customer” class

```
public class Point {  
    int x;  
    int y;  
    void move(int dx, int dy) { ... }  
}
```



# Instantiation

- The objects we create are instances of the class
  - ◊ The class is a blueprint of the object
- In Java, objects do not have pre-allocated storage
  - ◊ They are always dynamically created
  - ◊ They are stored on the heap

```
Point p = new Point();
```



## Example: texts and arrays

```
String    name    = "James Arthur Gosling";
String[]  names    = name.split(" ");
String    abbrev  = names[names.length-1] + ", "
                  + names[0].charAt(0) + ".";
```



# Textual representation

```

public class Person {
    private String family;
    private String middle;
    private String given;
    private boolean isWesternStyle;

    @Override
    public String toString() {
        char mid = middleName.charAt(0);
        String part1 = isWesternStyle ? given : family;
        String part2 = isWesternStyle ? family : given;
        return part1 + " " + mid + ". " + part2;
    }
}

```



## Textual representation: variants

```
@Override public String toString() {  
    // mid, part1, part2 as before  
  
    // Java 15+  
    return "%s %s. %s".formatted(part1, mid, part2);  
  
    // Java 5+  
    return String.format("%s %s. %s", part1, mid, part2);  
  
    // with efficient string concatenation  
    StringBuilder sb = new StringBuilder();  
    sb.append(part1);  
    sb.append(" ");  
    sb.append(mid);  
    sb.append(". ");  
    sb.append(part2);  
}
```



## Textual representation: details

It's not always printing

```
System.out.println(person.toString());
```

```
System.out.println(person);           // internally calls the above
```

```
String name = person.toString();
```

```
String parts = person.split(" ");
```

```
System.out.println(name + " starts with " + parts[0]);
```

"...".formatted(...) and String.format("...", ...) are related

```
String txt = "%s / %d = %f".formatted("seven", 2, 3.5);
```

```
System.out.println(txt);
```

```
System.out.printf("%s / %d = %f", "seven", 2, 3.5);
```





## Textual representation: details (2)

Beginner's mistake: don't use ==, use equals

```
String input = ...; // the user enters "he
System.out.println(input == "hello"); // wrong: false
System.out.println("hello" == input); // wrong: false
System.out.println(input.equals("hello")); // OK: true
System.out.println("hello".equals(input)); // OK: true
```

# Class, object, instantiation

## Point.java

```
public class Point { // class definition
    int x, y;         // fields
}
```

# Class, object, instantiation

## Point.java

```
public class Point { // class definition
    int x, y;         // fields
}
```

## Main.java

```
public class Main {
    public static void main(String[] args) {
        Point p = new Point(); // instantiation (on heap)
        p.x = 3;                // changing object state
        p.y = 3;                // changing object state
    }
}
```

# Compilation, execution

```
$ ls
Main.java  Point.java
$ javac *.java
$ ls
Main.class Main.java  Point.class  Point.java
$ java Point
Error: Main method not found in class Point, please define
the main method as:
    public static void main(String[] args)
$ java Main
$
```



Class, object, instance

# Stack and heap



# Fields: initialization

```
public class Point {  
    int x = 3, y = 3;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Point p = new Point();  
        System.out.println(p.x + " " + p.y);    // 3 3  
    }  
}
```





## Fields: initialization with default value

A zero-like value is assigned to fields automatically.

```

public class Point {
    int x, y = 3;
    // problem: it is difficult to see that x = 0,
    // better: initialize all fields on separate lines
}

public class Main {
    public static void main(String[] args) {
        Point p = new Point();
        System.out.println(p.x + " " + p.y); // 0 3 (NOT 3 3)
    }
}

```

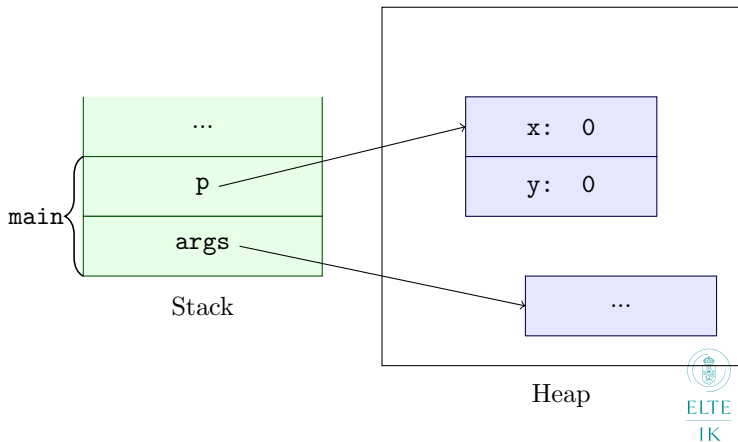


# Method

```
public class Point {  
    int x, y;    // initially 0 and 0  
    void move(/* Point this, */ int dx, int dy) {  
        // this: implicit parameter  
        this.x += dx;  
        this.y += dy;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Point p = new Point();  
        p.move(3,3); // p -> this, 3 -> dx, 3 -> dy  
    }  
}
```

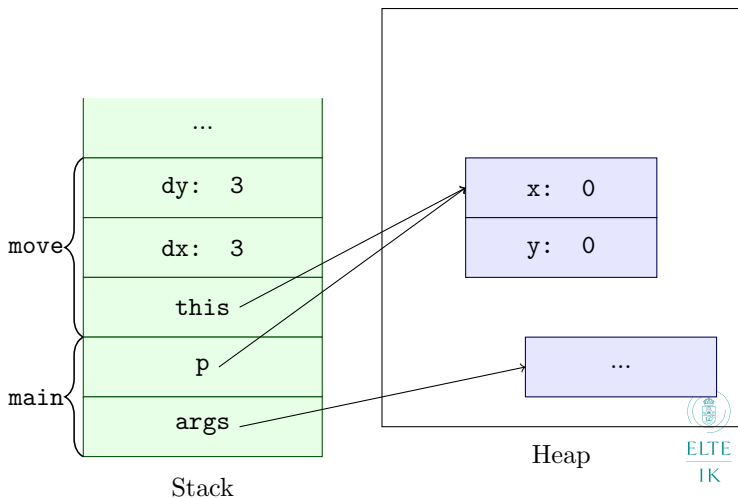
# Method activation record – 1

```
Point p = new Point();
```



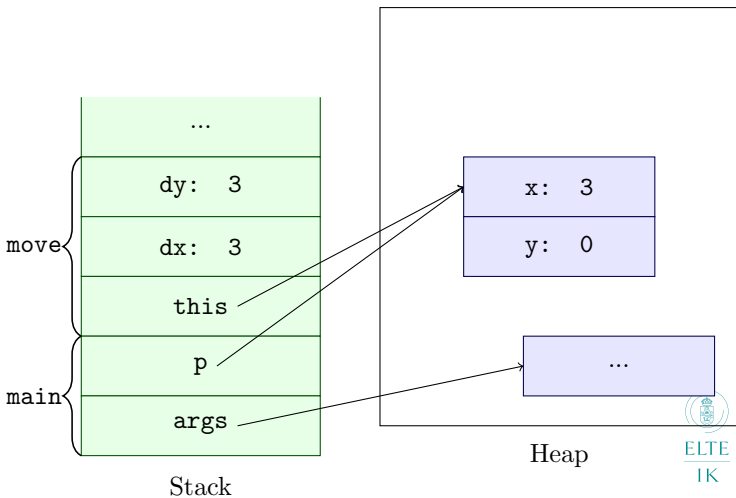
## Method activation record – 2

```
p.move(3,3);
```



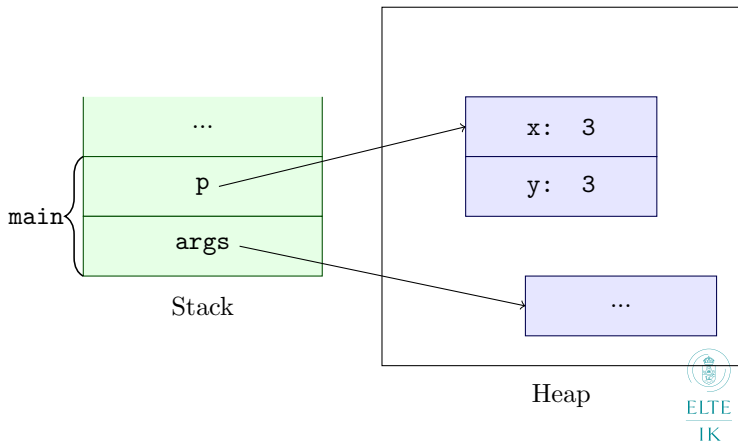
# Method activation record – 3

```
this.x += dx;
```



# Method activation record – 4

```
System.out.println(p.x + " " + p.y);
```



## You may leave out **this**

Here, this is implicit.

```
public class Point {  
    int x, y;    // 0, 0  
    void move(int dx, int dy) {  
        this.x += dx;  
        y += dy;    // equivalent to this.y = dy  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Point p = new Point();  
        p.move(3,3);  
    }  
}
```



# Initialization using a constructor

```

public class Point {
    int x, y;
    Point(/* Point this, */ int initialX, int initialY) {
        // this: implicit parameter
        this.x = initialX;
        this.y = initialY;
    }
}

class Main {
    public static void main(String[] args) {
        Point p = new Point(0,3);
        System.out.println(p.x + " " + p.y);    // 0 3
    }
}

```





# Initialization using a constructor without this

```

public class Point {
    int x, y;
    public Point(int initialX, int initialY) {
        x = initialX;
        y = initialY;
    }
}

class Main {
    public static void main(String[] args) {
        Point p = new Point(0,3);
        System.out.println(p.x + " " + p.y);    // 0 3
    }
}

```



## Name reuse

```

public class Point {
    int x, y;
    public Point(int x, int y) {    // name hiding
        this.x = x; // field x is not visible: arg x hides it
                        // this.x: qualified name
        this.y = y; // convention: give exactly the same name
                        // to related fields and args
    }
}

public class Main {
    public static void main(String[] args) {
        Point p = new Point(0,3);
        System.out.println(p.x + " " + p.y);    // 0 3
    }
}

```



# No-arg constructor

This constructor takes no parameters.

```
public class Point {
    int x, y;
    public Point() {}
}

public class Main {
    public static void main(String[] args) {
        Point p = new Point();
        System.out.println(p.x + " " + p.y);    // 0 0
    }
}
```



## Default constructor

```
public class Point {
    int x, y;
}

class Main {
    public static void main(String[] args) {
        Point p = new Point();
        System.out.println(p.x + " " + p.y);    // 0 0
    }
}
```

A no-arg constructor with an empty body is generated

```
public Point() {}
```



# Encapsulation

```
public class Time {
    int hour;
    int min;
    Time(int hour, int min) {
        this.hour = hour;
        this.min = min;
    }
    void aMinPassed() {
        if (min < 59) {
            ++min;
        } else { ... }
    } // (C) Monty Python
}
```

```
Time morning = new Time(6,10);
morning.aMinPassed();
int hour = morning.hour;
```

# Type invariant

```
public class Time {  
    int hour;                // 0 <= hour < 24  
    int min;                 // 0 <= min < 60  
    public Time(int hour, int min) {  
        this.hour = hour;  
        this.min = min;  
    }  
    void aMinPassed() {  
        if (min < 59) {  
            ++min;  
        } else { ... }  
    }  
}
```



# Creating a nonsense value

```
public class Time {
    int hour;
    int min;
    Time(int hour, int min) {
        this.hour = hour;
        this.min = min;
    }
    void aMinPassed() {
        if (min < 59) {
            ++min;
        } else { ... }
    }
}
```

```
Time morning = new Time(6,10);
morning.aMinPassed();
int hour = morning.hour;

morning.hour = -1;
morning = new Time(24,-1);
```

## Ensure type invariant on creation

```

public class Time {
    int hour;                // 0 <= hour < 24
    int min;                 // 0 <= min < 60
    public Time(int hour, int min) {
        if (0 <= hour && hour < 24 && 0 <= min && min < 60) {
            this.hour = hour;
            this.min = min;
        }
    }
    void aMinPassed() {
        if (min < 59) {
            ++min;
        } else { ... }
    }
}

```





# Avoid silent failure

```
public class Time {
    int hour;                // 0 <= hour < 24
    int min;                 // 0 <= min < 60
    public Time(int hour, int min) {
        if (0 <= hour && hour < 24 && 0 <= min && min < 60) {
            this.hour = hour;
            this.min = min;
        } else {
            throw new IllegalArgumentException("Wrong time!");
        }
    }
    void aMinPassed() { ... }
}
```

## Utility function

```

public class Time {
    ...
    public Time(int hour, int min) {
        if (isBetween(hour, 0, 24) && isBetween(min, 0, 60)) {
            this.hour = hour;
            this.min = min;
        } else {
            throw new IllegalArgumentException("Wrong time!");
        }
    }
    // utility function: makes the code easier to understand
    private boolean isBetween(int value, int min, int max) {
        return min <= value && value < max;
    }
}

```

# Early return

```
public class Time {  
    public Time(int hour, int min) {  
        // early return/throw: start by handling special cases  
        if (!isBetween(hour, 0, 24) || !isBetween(min, 0, 60)) {  
            throw new IllegalArgumentException("Wrong time!");  
        }  
  
        // "happy path": this is the common path  
        this.hour = hour;  
        this.min = min;  
    }  
  
    ...  
}
```



# Exception

- Occurs during runtime
- Indicates that execution continues in a non-usual way
  - ◇ Can signal some sort of problem
  - ◇ throw statement
- Possibly stops the program fully
- Can be handled in the program
  - ◇ try-catch statement



## Runtime error

```
public class Main {
    public static void main(String[] args) {
        public Time morning = new Time(24, -1);
    }
}
```

```
$ javac Time.java
$ javac Main.java
$ java Main
Exception in thread "main" java.lang.IllegalArgumentException:
Wrong time!
    at Time.<init>(Time.java:9)
    at Main.main(Main.java:3)
$
```



## Fields can be changed directly

```
public class Time {
    int hour;                // 0 <= hour < 24
    int min;                 // 0 <= min < 60
    ...
}
```

```
public class Main {
    public static void main(String[] args) {
        public Time morning = new Time(6,10);
        morning.aMinPassed();

        morning.hour = -1;    // ouch!
    }
}
```



private

## Hiding fields: private

```
public class Time {
    private int hour;           // 0 <= hour < 24
    private int min;           // 0 <= min < 60
    ...
}
```

```
public class Main {
    public static void main(String[] args) {
        public Time morning = new Time(6,10);
        morning.aMinPassed();

        morning.hour = -1;      // compilation error
    }
}
```

## Idiom: state is private, only changeable via methods

```

public class Time {
    private int hour;           // 0 <= hour < 24
    private int min;           // 0 <= min < 60
    public Time(int hour, int min) { ... }
    int getHour() { return hour; }
    int getMin() { return min; }
    void setHour(int hour) {
        if (0 <= hour && hour <= 23) {
            this.hour = hour;
        } else {
            throw new IllegalArgumentException("Wrong hour!");
        }
    }
    void setMin(int min) { ... }
    void aMinPassed() { ... }
}

```



## Convention: getters and setters

The field determines (almost) everything about getters/setters.

- name
- return type
- arguments
- body

```
public class Time {
    private int hour;           // 0 <= hour < 24
    public int getHour() { return hour; }
    public void setHour(int hour) {
        if (0 <= hour && hour <= 23) {
            this.hour = hour;
        } else {
            throw new IllegalArgumentException("Wrong hour!");
        }
    }
}
```



# Can change the representation

```
public class Time {
    private short mins;
    public Time(int hour, int min) {
        if (...) throw new IllegalArgumentException("Wrong time!");
        mins = 60*hour + min;
    }
    int getHour() { return mins / 60; }
    int getMin() { return mins % 60; }
    void setHour(int hour) {
        if (...) throw new IllegalArgumentException("Wrong hour!");
        mins = 60 * hour + getMin();
    }
    void setmin(int min) { ... }
    void aMinPassed() { ... }
}
```



# Information hiding

- Interface: make it lean
  - ◇ Allow only as much access to the object as necessary
  - ◇ This part is what is visible to the other classes
- Implementation details: make them inaccessible from the outside
  - ◇ Helper methods
  - ◇ Fields
- Advantages
  - ◇ Lets the class preserve its type invariant
  - ◇ Easier to evolve the code (representation change)
  - ◇ Loose coupling: other classes don't know about the internals
- Also desirable
  - ◇ Strong cohesion (the class has a single, well defined purpose)



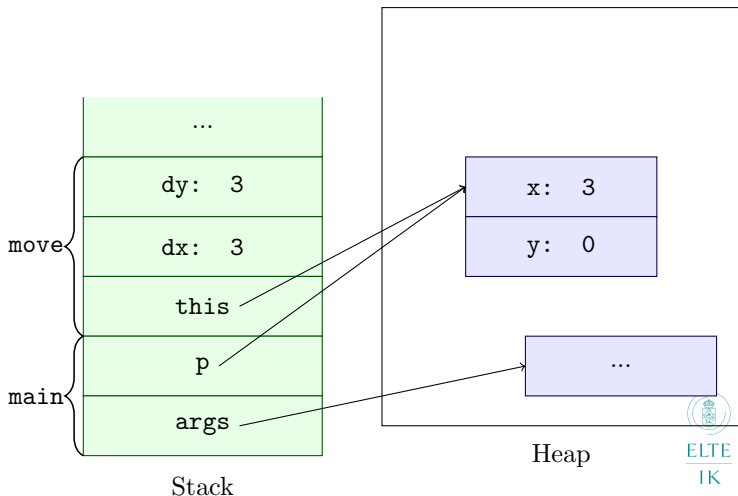
# Reference

```
Point p = new Point();  
p.x = 3;
```

- Its type is a class (not a primitive)
- Refers to an object
- Stored on the heap
- Creation: **new**
- Dereference: .



# Variables with different types in the memory



# Primitive/reference types

## Primitive types

- **byte**:  $[-128..127]$
- **short**:  $[-2^{15}..2^{15} - 1]$
- **int**:  $[-2^{31}..2^{31} - 1]$
- **long**: 8 bytes long
- **float**: 4 bytes long
- **double**: 8 bytes long
- **char**: 2 bytes long
- **boolean**: {**false**,**true**}

## Reference types

- Classes
- Array types
- ...



# Representation in the memory

## Runtime stack

Local variables and parameters  
(both primitives and references)

## Heap

Objects and their fields  
(both primitives and references)

Instance variable: data in an object that corresponds to a field.

## Scope and lifetime of variables

- Rules are similar to other languages such as C
- Lifetime of local variable: until the end of the scope
- Scope: from declaration to end of immediately containing block
- Hiding: only fields

```
public class Point {
    int x = 0, y = 0;
    void foo(int x) { // OK
        int y = 3;    // OK
        {
            int z = y;
            int y = x; // Compilation error
            ...
        }
    }
}
```



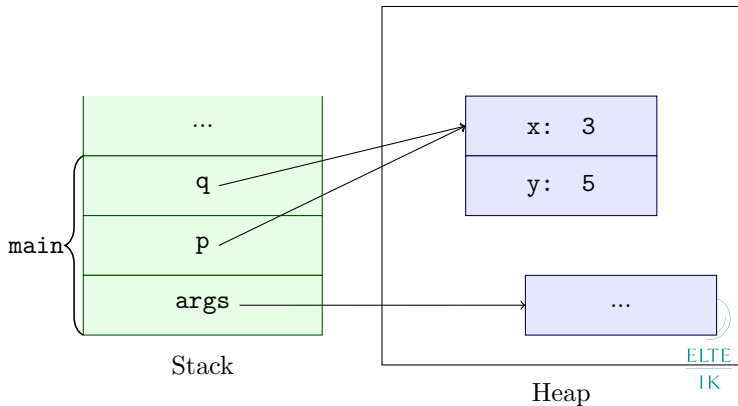
# Lifetime of objects

- Creation + initialization
- Pointing references at the new objects
  - ◇ Aliasing
- Garbage collection

```
new Point(3,5)
Point p = new Point(3,5);
Point q = p;
p = q = null;
```

# Aliasing

```
Point p = new Point(3,5), q = p;  
q.x = 6;
```





# Empty reference

```

Point p = null;
p = new Point(4,6);
if (p != null) {
    p = null;
}
p.x = 3;    // NullPointerException
  
```



## Empty reference

```
Point p = null;
p = new Point(4,6);
if (p != null) {
    p = null;
}
p.x = 3;    // NullPointerException
```

I call it my billion-dollar mistake.

— Tony Hoare, creator of **null**

## Empty reference

```
Point p = null;  
p = new Point(4,6);  
if (p != null) {  
    p = null;  
}  
p.x = 3;    // NullPointerException
```

I call it my billion-dollar mistake.

— Tony Hoare, creator of **null**

The dark side of the Force are they. Easily they flow, quick to join you in a fight. If once you start down the dark path, forever will it dominate your destiny, consume you it will... — Yoda

## Empty reference

```
Point p = null;  
p = new Point(4,6);  
if (p != null) {  
    p = null;  
}  
p.x = 3;    // NullPointerException
```

I call it my billion-dollar mistake.  
— Tony Hoare, creator of **null**

The dark side of the Force are they. Easily they flow, quick to join you in a fight. If once you start down the dark path, forever will it dominate your destiny, consume you it will... — Yoda





# Initialization: fields

Automatically to a zero-like value

```
public class Point {
    int x = 0, y = 0;
}
```

```
public class Point {
    int x, y;
}
```

```
public class Point {
    int x, y = 0;
}
```

```
public class Point {
    int x, y = x;
}
```



# Initialization: null reference

```
public class Hero {  
    String name;           // == null  
    Hero bestFriend;       // == null  
}
```

```
Hero ironMan = new Hero();  
ironMan.name = "Iron Man";  
// ironMan.bestFriend == null
```





# Initialization: local variables

- No automatic initialization
- Needs explicit assignment before it is used
  - ◊ Compilation error if missing (static semantic error)

```
public static void main(String[] args) {
    int i;
    Point p;
    p.x = i;    // compilation error for two reasons
}
```

# Guaranteed assignment

- On all execution paths, there must be an assignment before first use
- The compiler cannot check all corner cases (limited computability)

## Example from JLS (Chapter 16, Definite Assignment)

```
{  
    int k;  
    int n = 5;  
    if (n > 2) {  
        k = 3;  
    }  
    System.out.println(k); // k is not "definitely assigned"  
                           // before this statement  
}
```

# Static fields

- Similar to global variables in C
- Only one instance exists
- Use the class name to access it
  - ◇ No need to have an instance
- Stored in *static storage*, not inside the objects themselves

```
class Item {  
    static int counter = 0;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        System.out.println(Item.counter);  
    }  
}
```



## Instance variables and class variables

```
public class Item {
    static int counter = 0;
    int id = counter++;      // meaning: id = Item.counter++
}
```

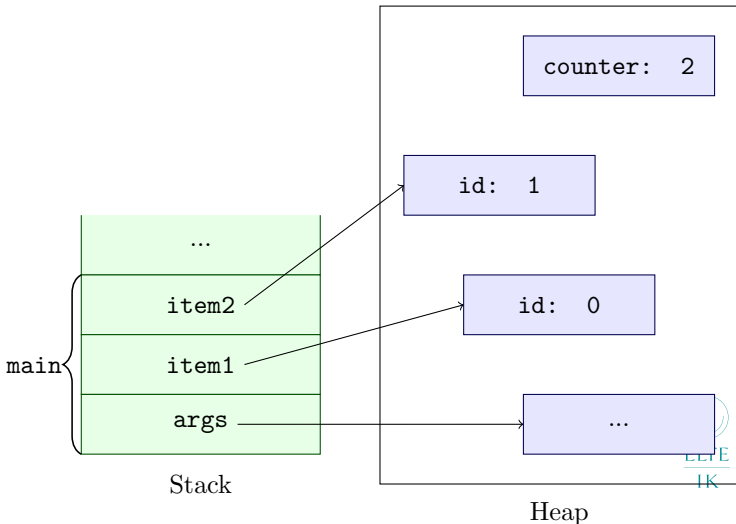
```
public class Main {
    public static void main(String[] args) {
        Item item1 = new Item(), item2 = new Item();
        System.out.println(item1.id);
        System.out.println(item2.id);

        System.out.println(Item.counter); // use it like this
        System.out.println(item1.counter); // ugly, never use
                                           // (Java allows it)
    }
}
```



## Static members

```
Item item1 = new Item(), item2 = new Item();
```





## Static methods

- Similar to global functions in C
- Can be called using the class
  - ◊ No need to have an instance
- Does not take the implicit parameter this
- Closely related to static fields

```
class Item {
    static int counter = 0;
    static void print() {
        System.out.println(counter);
    }
}

public class Main {
    public static void main(String[] args) {
        Item.print();
    }
}
```

## Static methods lack this

```
class Item {  
    static int counter = 0;  
    int id = counter++;  
    static void print() {  
        System.out.println(counter);  
        System.out.println(id); // cannot have a meaning  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Item.print();  
    }  
}
```

# Using **static** members

## Methods

- main must be **static**
- Can describe clean (“matematical”) computations
- Utility/helper methods

Fields: it is very limiting, only use when really necessary

- Dependency (coupling) between instances
- Harder to prove/test code correctness
- Harder to change the code
- Decreases encapsulation



# Garbage collection

Releases (frees) objects that are not needed anymore

## Correct

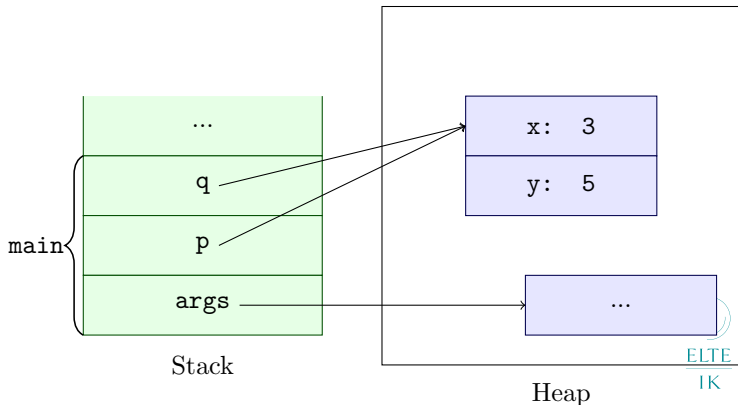
Only frees objects that are unreachable from the program

## Complete

Frees all unreachable objects

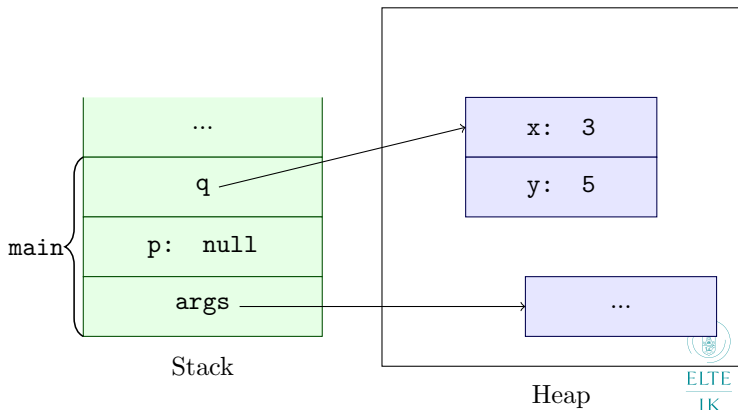
# Cannot be released yet

```
Point p = new Point(3,5);  
Point q = p;
```



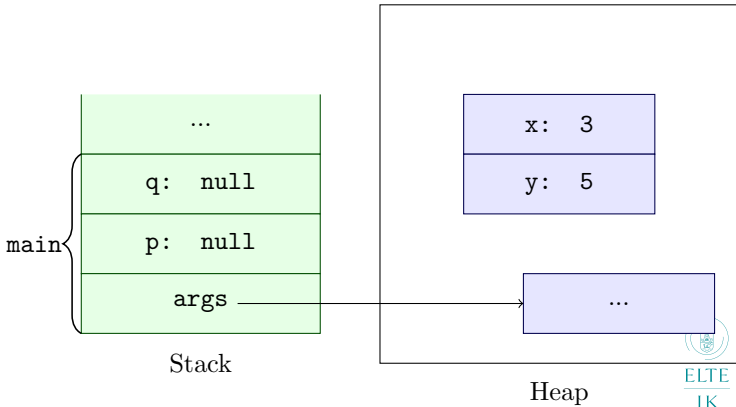
# Still cannot be released

`p = null;`



# Can be released now

q = null;





More complex example

## More complex example

```
public class Hero {
    String name;
    Hero bestFriend;
}
```

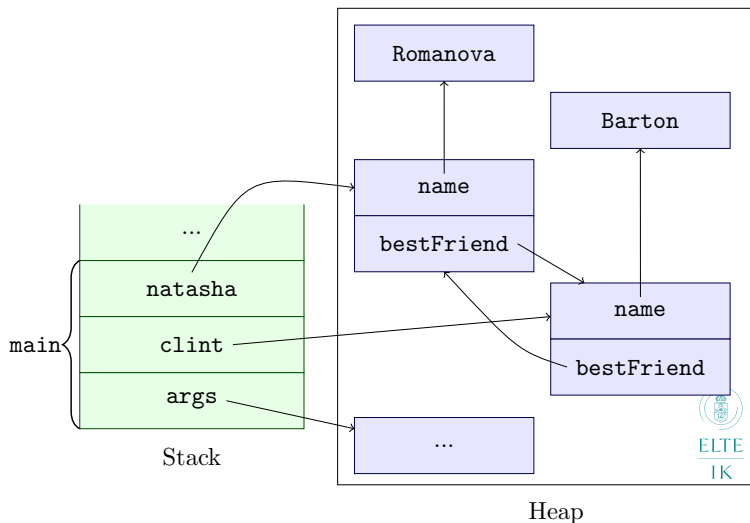
```
Hero clint    = new Hero();
Hero natasha  = new Hero();
```

```
clint.name      = "Barton";
natasha.name    = "Romanova";
clint.bestFriend = natasha;
natasha.bestFriend = clint;
```

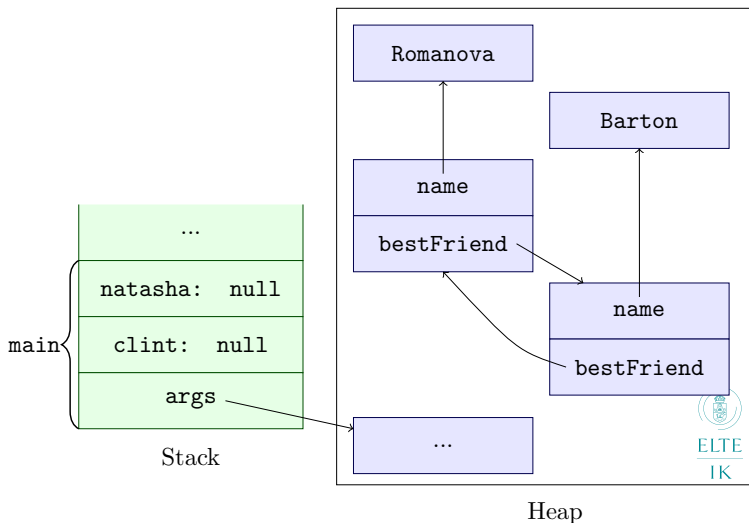
IK

More complex example

# Heroes in the memory



```
natasha = clint = null;
```



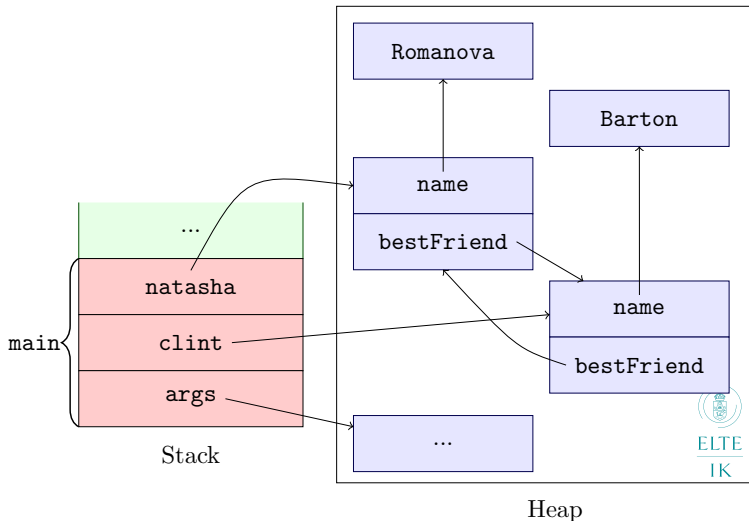


# Mark-and-Sweep garbage collection

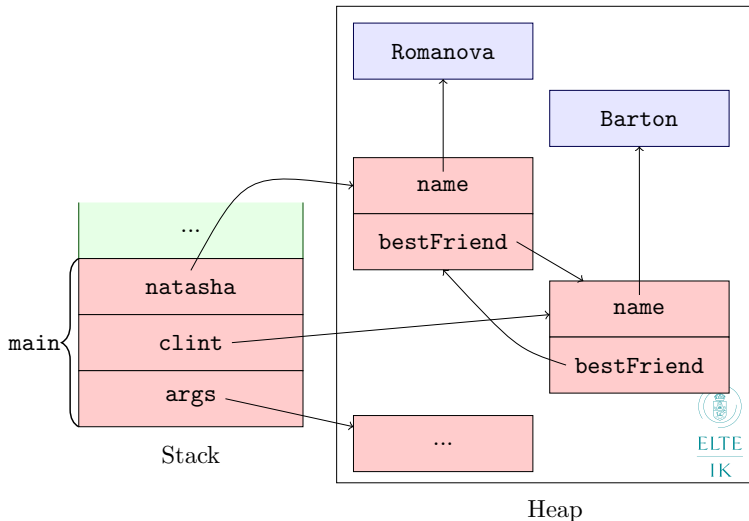
- Mark phase
  - ◇ Starting point: references on the heap
  - ◇ Mark all objects reachable from there
    - ▶ Continue marking objects reachable from those
    - ▶ ... until there are no more (transitive closure)
  - ◇ “Stop the world”: the program is paused
- Sweep phase
  - ◇ All unmarked objects can be freed now



# Mark-and-sweep: root set



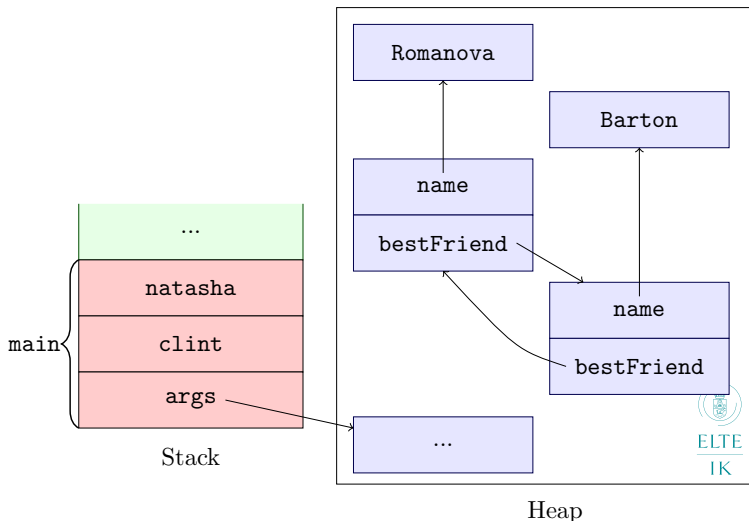
# Mark-and-sweep: propagation





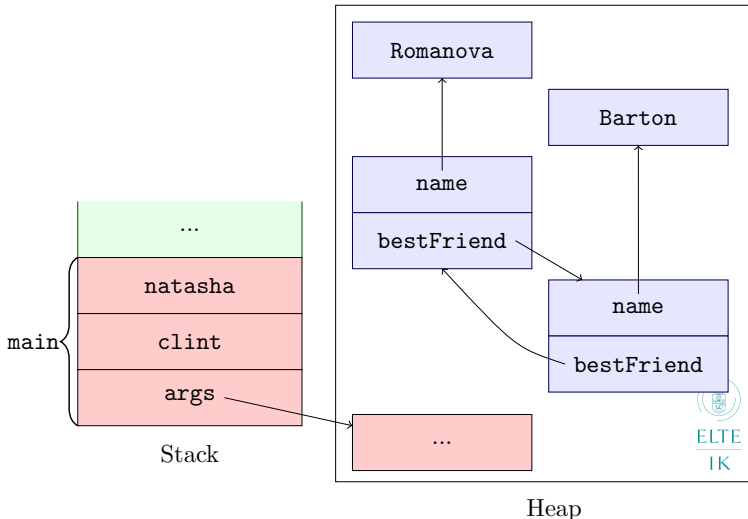
## Mark-and-Sweep GC

Mark-and-sweep: `natasha = clint = null`; again  
`natasha = clint = null`;



## Mark-and-sweep: end of the mark phase

- The sweep phase frees the unreachable objects





# Array

- Data structure
- Elements one after the other in sequence
- Indexing in constant time
- First index is 0



# Array types

`String[] args`

- `args` is a reference
- Arrays are objects
  - ◊ Stored on the heap
  - ◊ Creation: **new**
- Arrays know their size, e.g. `args.length`
- Indexing: runtime check
  - ◊ `ArrayIndexOutOfBoundsException`



# Iterating arrays

```
public static void main(String[] args) {  
    for (int i = 0; i < args.length; ++i) {  
        System.out.println(args[i]);  
    }  
}
```





# ArrayIndexOutOfBoundsException

```
public static void main(String[] args) {  
    for (int i = 0; i <= args.length; ++i) {  
        System.out.println(args[i]);  
    }  
}
```



# Enhanced for loop

```
public static void main(String[] args) {
    for (int i = 0; i < args.length; ++i) {
        System.out.println(args[i]);
    }
}
```

```
public static void main(String[] args) {
    for (String arg : args) {
        System.out.println(arg);
    }
}
```



# Create, init, sort

```
public class Sort {
    public static void main(String[] args) {
        int[] numbers = new int[args.length]; // all zeros

        for (int i = 0; i < args.length; ++i) {
            numbers[i] = Integer.parseInt(args[i]);
        }

        java.util.Arrays.sort(numbers);

        for (int n: numbers) { System.out.println(n); }
    }
}
```



# Import

```

import java.util.Arrays;

public class Sort {
    public static void main(String[] args) {
        int[] numbers = new int[args.length];

        for (int i = 0; i < args.length; ++i) {
            numbers[i] = Integer.parseInt(args[i]);
        }

        Arrays.sort(numbers);

        for (int n: numbers) { System.out.println(n); }
    }
}

```



# Static import

```
import static java.util.Arrays.sort;

public class Sort {
    public static void main(String[] args) {
        int[] numbers = new int[args.length];

        for (int i = 0; i < args.length; ++i) {
            numbers[i] = Integer.parseInt(args[i]);
        }

        sort(numbers);

        for (int n: numbers) { System.out.println(n); }
    }
}
```

## Variable arity methods (vararg)

```
private static int[] getInts(String... txts) {  
    int[] retval = new int[txts.length];  
    for (int i = 0; i < txts.length; ++i) {  
        retval[i] = Integer.parseInt(txts[i]);  
    }  
    return retval;  
}  
  
public static void main(String... args) {  
    int[] nums = getInts("1", "2", "3");  
    // = getInts(new String[]{ "1", "2", "3" });  
    // = getInts("1 2 3".split(" "));  
    // = getInts(args);  
    sort(nums);  
    for (int num: nums) { System.out.println(num); }  
}
```



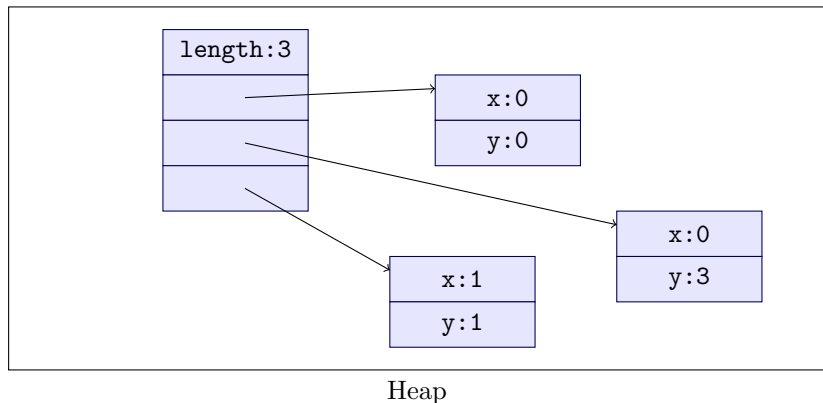
# Array of references

```
Point[] triangle = { new Point(0,0),  
                      new Point(0,3),  
                      new Point(1,1) };
```



## Array of references

```
Point[] triangle = { new Point(0,0),
                     new Point(0,3),
                     new Point(1,1) };
```





# Step by step

```
static void walk() {  
    Foot[] centipede;  
    // error: The local variable centipede may not have been initi  
    System.out.println(centipede.length);  
}
```

# Step by step

```
static void walk() {  
    Foot[] centipede;  
    // error: The local variable centipede may not have been initi  
    System.out.println(centipede.length);  
  
    centipede = null;  
    System.out.println(centipede.length);  
}
```



# Step by step

```
static void walk() {
    Foot[] centipede;
    // error: The local variable centipede may not have been initi
    System.out.println(centipede.length);

    centipede = null;
    System.out.println(centipede.length);

    centipede = new Foot[100];
    System.out.println(centipede.length);
}
```

# Step by step

```
static void walk() {  
    Foot[] centipede;  
    // error: The local variable centipede may not have been initi  
    System.out.println(centipede.length);  
  
    centipede = null;  
    System.out.println(centipede.length);  
  
    centipede = new Foot[100];  
    System.out.println(centipede.length);  
  
    for (int i = 0; i<100; i+=2) {  
        centipede[i]    = new Foot("left");  
        centipede[i+1] = new Foot("right");  
    }  
}
```



More than one dimension

# Matrix

```
double[][] id3 = { {1,0,0}, {0,1,0}, {0,0,1} };
```



# Matrix

```
double[][] id3 = { {1,0,0}, {0,1,0}, {0,0,1} };
```

```
static double[][] id(int n) {
    double[][] matrix = new double[n][n];
    for (int i=0; i<n; ++i) {
        matrix[i][i] = 1;
    }
    return matrix;
}
```

## C versus Java

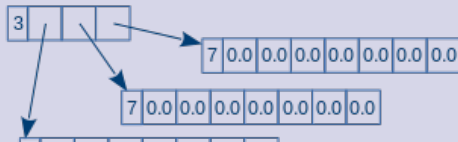
## C: multidimensional array

```
double matrix[3][7];  
for (int i=0; i<3; ++i)  
    for (int j=0; j<7; ++j)  
        matrix[i][j] = 0.0;
```



## Java: array of arrays

```
double[][] matrix =  
    new double[3][7];
```





# Indexing

## C: array with 3 dimensions

```
T t[L][M][N];
```

$$\text{addr}(t_{i,j,k}) = \text{addr}(t) + ((i \cdot M + j) \cdot N + k) \cdot \text{sizeof}(T)$$

## Java: array of arrays of arrays

```
T[][][] t = new T[L][M][N];
```

$$\text{addr}(t_{i,j,k}) = \text{val}_8 \left( \text{val}_8(\text{addr}(t) + 4 + i \cdot 8) + 4 + j \cdot 8 \right) + 4 + k \cdot \text{sizeof}(T)$$

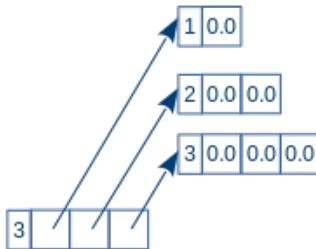
ELTE  
IK





## Lower triangular matrix

```
static double[][] zeroLowerTriangular(int n) {
    double[][] result = new double[n][];
    for (int i = 0; i < n; ++i) {
        result[i] = new double[i+1];
    }
    return result;
}
```



# Command line arguments

- In Java: `String[] args`
- In C: `char *argv[]`
  - ◇ Java equivalent: `char[][] argv`

# Reference types in Java

- **class**
  - ◇ enumeration types (**enum**)
- **interface**
- annotation types (**@interface**)



# Enumeration type

```
enum Day { SUN, MON, TUE, WED, THU, FRI, SAT }
```

- It is a reference type
- Its values are objects, not simple ints

# Enumeration type

```
enum Day { SUN, MON, TUE, WED, THU, FRI, SAT }
```

- It is a reference type
- Its values are objects, not simple ints

```
Day best = Day.SAT;           // also possible to "import static"
best = 3;                     // compilation error
int n = best;                 // compilation error
int m = best.ordinal();      // 6
```

- The values are all created at the definition site
- No way to create further instances
  - ◊ The constructor cannot be called later on: ~~new Day()~~



It's a special class in Java

# Constructors, members

```
public enum Coin {
    PENNY(1), NICKEL(5), DIME(10), QUARTER(25);

    private final int centValue;

    Coin(int centValue) { this.centValue = centValue; }

    public int centValue() { return centValue; }

    public int percentageOf(Coin that) {
        return 100 * centValue / that.centValue();
    }
} // Source: Java Community Process (modified)
```



It's a special class in Java

## Predefined members

```
public enum Coin {
    PENNY(1), NICKEL(5), DIME(10), QUARTER(25); ...
}

Coin[] allCoins = Coin.values();
System.out.println(allCoins.length);           // 4
System.out.println(allCoins[0] == Coin.PENNY); // true
System.out.println(allCoins[3] == Coin.QUARTER); // true

String txt = Coin.NICKEL.toString();
System.out.println(txt);                        // NICKEL

Coin coin = Coin.valueOf("DIME");
System.out.println(coin == Coin.DIME);         // true

System.out.println(Coin.PENNY.ordinal());      // 0
System.out.println(Coin.QUARTER.ordinal());    // 3
```



# In a switch statement and expression

```
static int workingHours(Day day) {  
    switch (day) { // switch statement  
        case SUN: case SAT: return 0;  
        case FRI:      return 6;  
        default:       return 8;  
    }  
}
```



# In a switch statement and expression

```
static int workingHours(Day day) {  
    switch (day) { // switch statement  
        case SUN: case SAT: return 0;  
        case FRI:      return 6;  
        default:       return 8;  
    }  
}
```

```
static int workingHours(Day day) {  
    return switch (day) { // Java 12+: switch expression  
        case SAT, SUN -> 0;  
        case FRI      -> 0;  
        default       -> 2;  
    };  
}
```